

Psychology 213
Human Information Processing and Artificial Intelligence
Assignment 3
Winning at Random-Number Texas Hold'Em

20 Points

Due Tues, Nov 25, 11:59 p.m. Eastern Time:

In the Assignment3_Class zip file you will find a set of Python files concerned with playing RandomNumber Texas Hold'Em. The file tester.py is the test code. It will load the obscured code of 90 players created by past students (P001 through P179, odd numbers only) as a set of opponents. It will also load the code for two “dummy” players, **John1** and **AllIn**, plus the obscured code for the reference program **John3**.

The game has been simplified such that you do not need to have any knowledge of poker cards. It is also the case that it only has to deal with two-player games. Finally, there are not any common cards (which makes it particularly unlike real Texas Hold'Em) but rather each player gets a single card and bets on that. The rules are:

1. Each player starts with 100 points
2. The players alternate between the roles of Small Blind and Big Blind.
3. Small Blind opens by betting 1 point.
4. Big Blind opens by betting 2 points.
5. Then they are each dealt a single card that contains a number uniformly distributed between 0 and 1. Since they do not see their opponent's card this card reflects that player's probability of having the winning hand.
6. Small Blind can choose to call (match) or raise Big Blind's bet. If not, Small Blind folds and loses its 1-point bet.
7. A bet less than the current bet is interpreted as a fold, and the current bet is lost.
8. If Small Blind calls (making their bet 2 points), the Big Blind can raise or do nothing. If Big Blind does not raise, the betting is over. If the Small Blind raised, the Big Blind can choose to fold, call, or raise.
9. The betting only continues if the previous player raises, and the current player can then fold, call, or raise.
10. If one player folds, that player loses the amount bet and neither player sees the other's cards.
11. If the game ends in a call, the players see each other's cards and the highest card wins.
12. Every 100 hands the minimum bet by Small Blind doubles, as does Big Blind (2x Small Blind).
All raises must be in multiples of the Small Blind's minimum.
13. The maximum bet that can be made is the amount possessed by the player with the least points.
If either player bets more than this it is interpreted as the maximum.
14. The game ends when one player has won all 200 points.

You are to write a player class, using pokerplayer.py as a starting point. For examples, see the **John1** and **AllIn** classes in opponents/basic_players.py. There are 4 function calls your player must handle. Common arguments passed to these functions are: **card**, **myscore**, **oppscore**, **minbet**, and **pot**. **Card** is the value of its card (except at the end of the game when it may be the opponent's card), **myscore** is its number of points, **oppscore** is the opponent's score, **minbet** is the current minimum bet, and **pot** is the current bet. The 4 functions are:

1. **__init__()**: This gives you an opportunity to set up some structure before entering a series of games against an opponent. If you choose to use global variables, it is probably wise to reset them between opponents.
2. **start(blind, card, ...)**: This is called at the beginning of the game informing the program of its blind (big or small) and its card.
3. **bet(...)**: This is a request to make a bet. NOTE the response to this is to specify a total amount the player will put into the pot, NOT an incremental amount added to your current bet. A number less than the current opponent bet is treated as a fold.
4. **end(iwon, oppcard, ...)**: This informs the program of the game result (a win or a loss). If the game ended with someone folding then **oppcard** is None. Otherwise **oppcard** will contain the opponent's card.

You have the 90 past students to test your program against, but it will be graded according to how well it does against the 89 even-numbered past student programs that have been held out. Below is a call of AllIn against the 90 practice students with 200 games per student:

```
play(AllIn,training_opponents,200,0)
Score is 0.304.
Time to run is 3.56 seconds.
0.3041111111111111
```

Your program will be graded according to its performance on 200 games per opponent. The fourth argument, 0 in this case, controls how much is printed out and 0 is the briefest option. Below is an example of its play with some detail (fourth argument is 1) against just the first 5 opponents for brevity:

```
play(John1, training_opponents[0:5],200,1)
John1 won 54% against P001
John1 won 62% against P003
John1 won 20% against P005
John1 won 93% against P007
John1 won 65% against P009
Score is 0.591.
Time to run is 0.34 seconds.
0.591
```

As you can confirm with the obscured code, the reference solution **John3** tends to score about 71%-72%

```
play(John3,training_opponents,200,0)
Score is 0.716.
Time to run is 40.36 seconds.
0.7156666666666667
```

The results also print out run times. Keep in mind that the run time are determined by both your program and the opponent. **AllIn** runs so quickly because its simple strategy means matches can be resolved rapidly.

Grading

Your performance will be converted to a grade out of 14. As before if a student can beat my score, they will receive a perfect grade and grades will be extrapolated down from there. The students' players will also be entered into a tournament against each other. The best scoring program will get 3/3 and the worst 0/3. Finally, there will be 3 points for a 1-2 page document discussing what this your solution and/or the assignment reflects and/or does not reflect about human decision-making and communication

(2 points) and what use you made of Generative AI and how it compares to human processing (1 point). The maximum score for this assignment is 20 points.

Submission

Submit your **pokerplayer.py** and exported log of your interactions with GenAI in a zip folder. Submit your discussion as PDF separately from the zipped folder. Please also include your name and Andrew id in the text file. Submit your **pokerplayer.py** solution and the discussion separately.

Your solution should be contained by pokerplay.py, without changing any other files.

GenAI Reporting

You may decide the specific ways in while you use GenAI in this assignment. Ensure you export the log file of your interactions with Gemini or the GenAI tool of your choosing.

If you do not wish to use GenAI to complete the assignment, you may upload the starter code to Gemini and compare its solution to your own.